

# What is VIM

Vim (Vi IMproved) is a highly configurable, powerful text editor that is an enhanced version of the older vi editor<sup>[1] [2]</sup>. It was created by Bram Moolenaar in 1991 as a clone of Bill Joy's vi for the Amiga computer<sup>[2]</sup>. Vim is designed for use both from a command-line interface and as a standalone application in a graphical user interface<sup>[2]</sup>.

## Key Features

- **Modal editing:** Vim uses different modes for text input and command operations, allowing for efficient editing without leaving the home row of the keyboard<sup>[1] [3]</sup>.
- **Customization:** Vim is highly configurable, supporting hundreds of programming languages and file formats<sup>[1]</sup>.
- **Extensibility:** Users can enhance Vim's functionality through a wide variety of plugins<sup>[1]</sup>.
- **Cross-platform:** Available on Unix systems, MS-Windows, macOS, and other platforms<sup>[1] [2]</sup>.

## Advantages

1. **Power:** Vim offers powerful search and replace functions<sup>[1]</sup>.
2. **Efficiency:** Its modal nature and keyboard-centric approach allow for fast text manipulation<sup>[3]</sup>.
3. **Ubiquity:** Vim is present on most Unix-like systems, making it a valuable skill for system administrators and developers<sup>[4]</sup>.
4. **Low resource usage:** Vim has a small memory footprint and can run in a terminal<sup>[4]</sup>.
5. **Command-centered:** Complex text-related tasks can be performed with a few commands<sup>[4]</sup>.

Vim's learning curve can be steep, but many users find the investment worthwhile due to the significant productivity gains it offers once mastered<sup>[3] [4]</sup>.

✱

# What are the different modes in VIM?

Vim (Vi IMproved) is a powerful text editor that uses different modes for various editing tasks. The main modes in Vim are:

## Normal Mode

This is the default mode when you open Vim. In Normal mode, you can:

- Navigate through the text
- Execute commands
- Perform operations like copying, cutting, and pasting<sup>[5]</sup> <sup>[6]</sup>

To enter Normal mode from other modes, press the `Esc` key.

## Insert Mode

Insert mode allows you to type and modify text directly, similar to most modern text editors<sup>[5]</sup> <sup>[7]</sup>. To enter Insert mode from Normal mode:

- Press `i` to insert text at the cursor position
- Press `a` to append text after the cursor
- Press `o` to insert a new line below the current line<sup>[5]</sup>

## Visual Mode

Visual mode is used for selecting and manipulating blocks of text<sup>[5]</sup> <sup>[6]</sup>. There are three types of Visual mode:

1. Character-wise (press `v`): Select individual characters
2. Line-wise (press `V`): Select entire lines
3. Block-wise (press `Ctrl+v`): Select rectangular blocks of text<sup>[6]</sup>

## Command-line Mode

This mode is used for entering Vim commands. To enter Command-line mode:

- Press `:` to enter commands
- Press `/` to search forward
- Press `?` to search backward<sup>[5]</sup> <sup>[8]</sup>

## Replace Mode

Replace mode allows you to overwrite existing text. To enter Replace mode:

- Press `R` for multi-character replace
- Press `x` to replace a single character<sup>[7]</sup> <sup>[9]</sup>

These modes are fundamental to Vim's functionality and efficiency. By switching between modes, users can perform complex editing tasks with minimal keystrokes, making Vim a powerful tool for text editing and programming.

# How to navigate through text?

Navigating through text in Vim is primarily done in Normal mode using various keyboard commands. Here are some essential navigation techniques:

## Basic Movement

- **h**: Move left
- **j**: Move down
- **k**: Move up
- **l**: Move right

## Word Navigation

- **w**: Move to the beginning of the next word
- **b**: Move to the beginning of the previous word
- **e**: Move to the end of the current/next word

## Line Navigation

- **0**: Move to the beginning of the line
- **^**: Move to the first non-blank character of the line
- **\$**: Move to the end of the line

## Document Navigation

- **gg**: Go to the first line of the document
- **G**: Go to the last line of the document
- **:n**: Go to line number n (e.g., :10 goes to line 10)

## Screen Navigation

- **Ctrl+f**: Move forward one full screen
- **Ctrl+b**: Move backward one full screen
- **Ctrl+d**: Move forward half a screen
- **Ctrl+u**: Move backward half a screen

## Specific Character Navigation

- **f{char}**: Move to the next occurrence of {char} on the current line
- **F{char}**: Move to the previous occurrence of {char} on the current line
- **t{char}**: Move to just before the next occurrence of {char}
- **T{char}**: Move to just after the previous occurrence of {char}

## Paragraph and Block Navigation

- **}**: Move to the next paragraph
- **{**: Move to the previous paragraph
- **%**: Move to the matching parenthesis, bracket, or brace

These commands can be combined with numbers for repetition. For example, **5j** moves down 5 lines, and **3w** moves forward 3 words. Mastering these navigation commands can significantly improve your editing efficiency in Vim.

## How to cut, copy or paste stuff?

Vim uses specific commands for cutting, copying, and pasting text, which are performed primarily in Normal mode. Here's how to perform these operations:

### Copying (Yanking)

- **yy**: Copy the current line
- **3yy**: Copy three lines, starting from the cursor position
- **y\$**: Copy from the cursor to the end of the line
- **yw**: Copy to the start of the next word
- **yiw**: Copy the current word

### Cutting (Deleting)

- **dd**: Cut (delete) the current line
- **3dd**: Cut three lines, starting from the cursor position
- **d\$**: Cut from the cursor to the end of the line
- **dw**: Cut to the start of the next word

### Pasting (Putting)

- **p**: Paste after the cursor
- **P**: Paste before the cursor

### Using Visual Mode

1. Press **v** to enter Visual mode
2. Use movement keys to select text
3. Press **y** to copy or **d** to cut
4. Move the cursor to the desired location
5. Press **p** to paste after the cursor or **P** to paste before

These commands allow for efficient text manipulation in Vim. Remember that Vim uses its own terminology: copy is called "yank," cut is called "delete," and paste is called "put" <sup>[10]</sup> <sup>[11]</sup>.



# how to cut/copy to clipboard

To cut or copy text to the clipboard in Vim, you can use the following methods:

## Using the System Clipboard

1. Ensure your Vim has clipboard support by checking if `vim --version` shows `+clipboard`.
2. Use the `+` or `*` register to interact with the system clipboard:
  - To copy: `"+y` or `"*y` (followed by a motion)
  - To cut: `"+d` or `"*d` (followed by a motion)

For example:

- `"+yy` copies the current line to clipboard
- `"+dd` cuts the current line to clipboard

## Visual Mode Selection

1. Press `v` to enter Visual mode
2. Select the desired text
3. Press `"+y` to copy or `"+d` to cut to clipboard

## Clipboard Configuration

Add `set clipboard=unnamed` or `set clipboard=unnamedplus` to your `.vimrc` file to make all yank, delete, and put operations interact with the system clipboard automatically [\[12\]](#) [\[13\]](#).

## Alternative Methods

- For X11 systems without clipboard support, use: `:w !xclip -sel clip` to pipe selection to `xclip` [\[13\]](#).
- On macOS, you can use: `:w !pbcopy` to copy the current buffer to clipboard [\[13\]](#).

Remember that the exact behavior may vary depending on your operating system and Vim configuration.



# How to go to terminal ?

In Vim, there are several ways to access the terminal without closing your current editing session. Here are the main methods:

## Using Built-in Terminal (Vim 8.1+)

1. Open a terminal window within Vim:  
`:term` OR `:terminal`
2. Switch between the terminal and your file:
  - Use `Ctrl-W N` to enter Normal mode in the terminal
  - Use `Ctrl-W w` to cycle between windows

## Suspending Vim

1. Suspend Vim and return to your shell:  
Press `Ctrl-Z`
2. Resume Vim:  
Type `fg` in your shell and press Enter

## External Command Execution

1. Run a single shell command:  
`:!command`  
For example, `:!ls` to list files
2. Open an interactive shell:  
`:sh` OR `:shell`
3. Return to Vim:  
Type `exit` in the shell and press Enter

## Using Tmux (External Tool)

1. If you're using Tmux, you can create a new terminal pane:  
Press `Ctrl-B` then `"` (split horizontally) or `%` (split vertically)
2. Switch between panes:  
`Ctrl-B` then arrow keys

These methods allow you to access the terminal while keeping your Vim session open, enhancing your workflow efficiency.

1. <https://datascientest.com/en/vim-the-comprehensive-guide-to-this-text-editor>
2. [https://en.wikipedia.org/wiki/Vim\\_\(text\\_editor\)](https://en.wikipedia.org/wiki/Vim_(text_editor))
3. <https://www.linuxfoundation.org/blog/blog/classic-sysadmin-vim-101-a-beginners-guide-to-vim>
4. <https://www.loginradius.com/blog/engineering/vim-getting-started/>
5. <https://www.warp.dev/terminus/vim-modes>
6. <https://builtin.com/articles/vim-visual-mode>
7. <https://www.inmotionhosting.com/support/server/ssh/vim/>
8. <https://www.redhat.com/en/blog/four-things-vim>
9. <https://unix.stackexchange.com/questions/57705/what-is-a-modeless-vs-a-modal-editor>

10. <https://linuxize.com/post/how-to-copy-cut-paste-in-vim/>
11. <https://tosbourn.com/copy-paste-vim/>
12. <https://vi.stackexchange.com/questions/84/how-can-i-copy-text-to-the-system-clipboard-from-vim>
13. <https://stackoverflow.com/questions/3961859/how-to-copy-to-clipboard-in-vim>